

EVENT MANAGEMENT SYSTEM

Prepared By:

Ibrahim Issa

Ibrahim Haidar



Event Management System

This report details a fully implemented Event Management System, a comprehensive database project designed to model real-world event operations. It covers venue scheduling, organizer coordination, session management, speaker participation, ticket sales, attendee records, and sponsor involvement. The system demonstrates mastery of conceptual design, logical modeling, SQL implementation, query formulation, and relational reasoning, providing a robust solution for managing diverse event aspects.

Project Objectives

The primary objectives for this project were meticulously defined to ensure a comprehensive and functional system.



ER/EER Model Design

To design a complete Entity-Relationship (ER) and Extended Entity-Relationship (EER) model that accurately represents the entire event lifecycle, capturing all entities and their relationships.



Normalized Relational Schema

To construct a normalized relational schema, ensuring data integrity and efficiency, complete with appropriate constraints to maintain data consistency.



SQL Implementation & Data Population

To implement the designed schema using SQL, populating it with meaningful sample data to facilitate testing and demonstration of functionality.



Diverse SQL Query Development

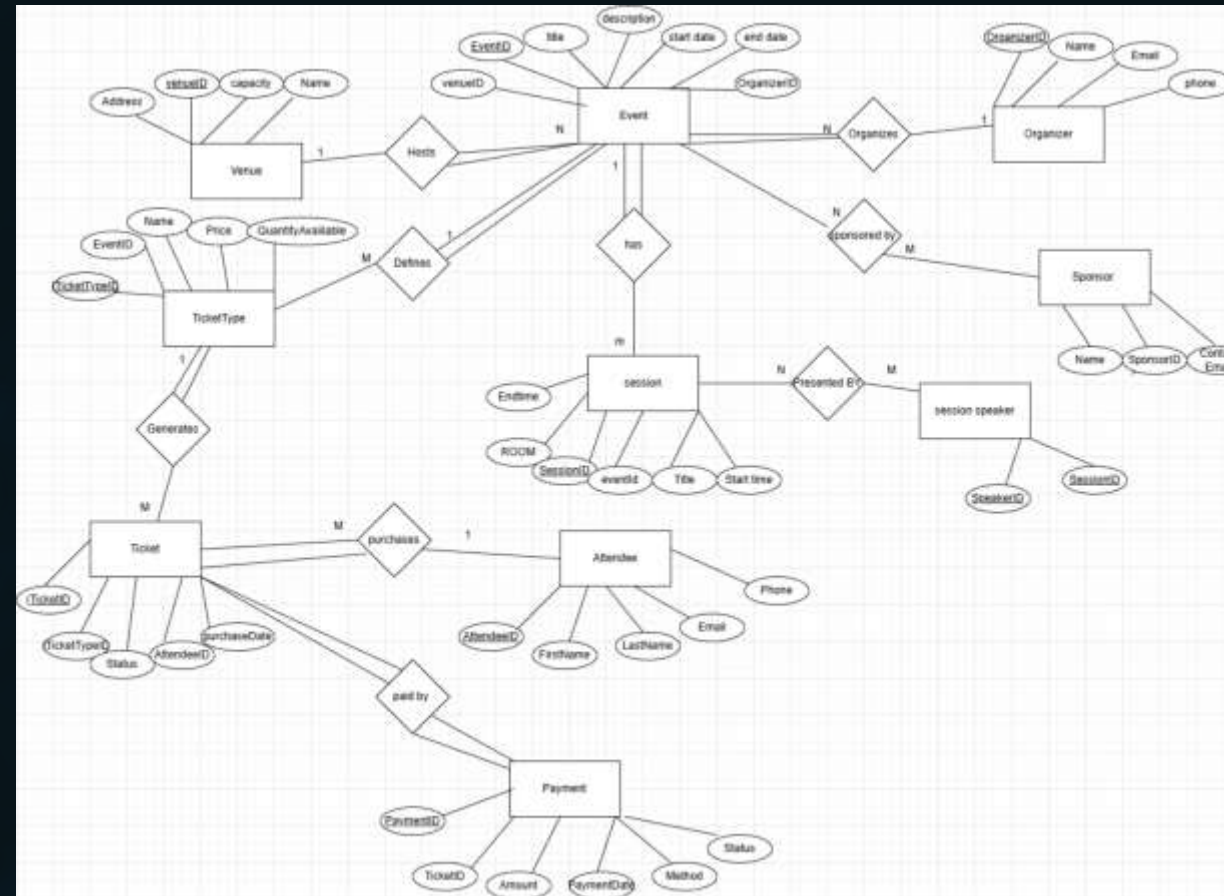
To develop and execute at least fifteen diverse SQL queries, showcasing the system's capabilities in retrieving and manipulating event-related data.



Professional Technical Documentation

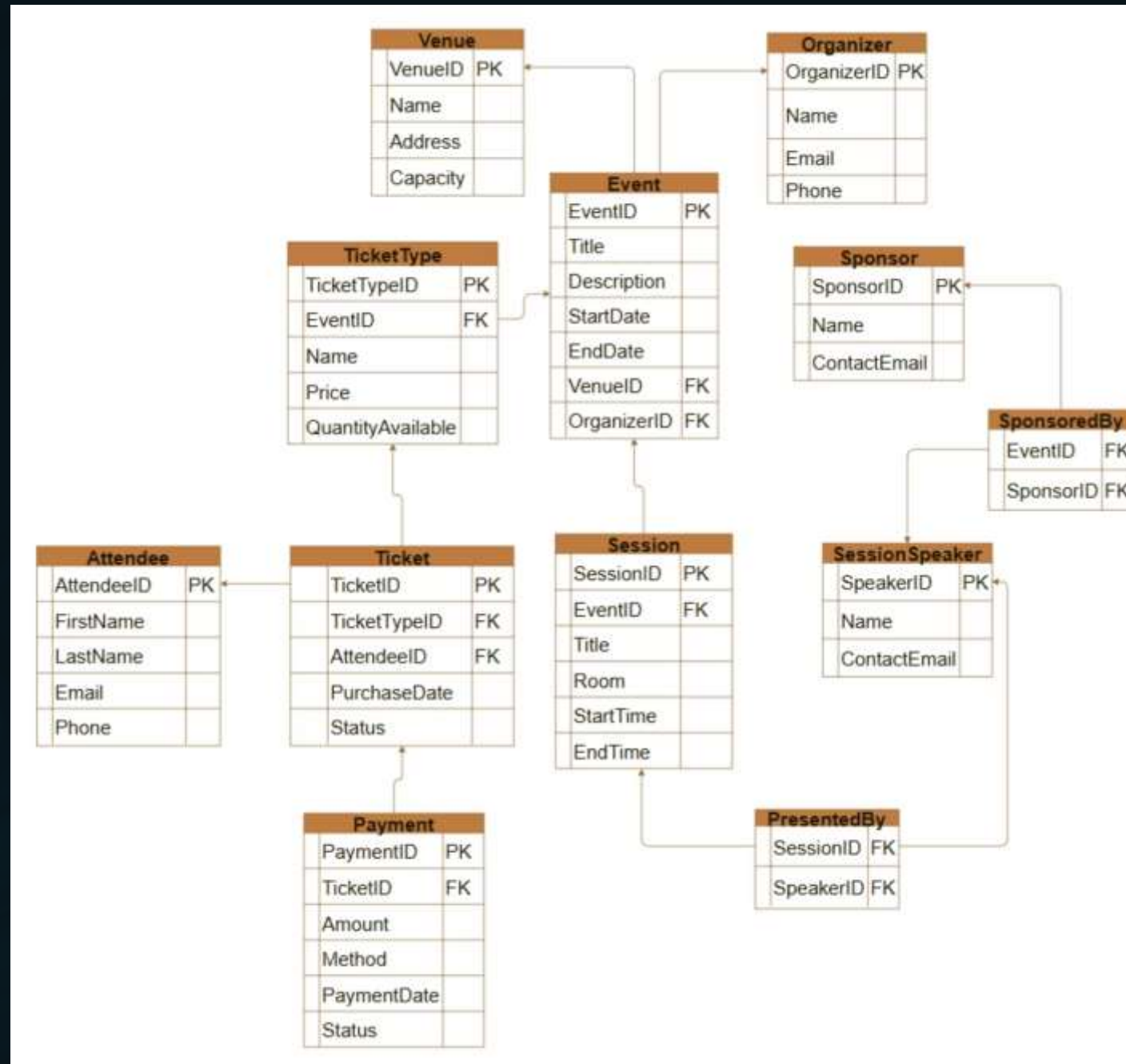
To deliver a polished and professional technical document detailing the full system, including functional dependencies and relational algebra expressions for selected queries.

Database Design: ER/EER Diagram



ER/EER Model Design Explanation: The ER/EER model represents all main entities of the Event Management System, including Event, Venue, Organizer, TicketType, Ticket, Attendee, Payment, Session, SessionSpeaker, and Sponsor. Relationships such as Event–Session, Event–TicketType, and TicketType–Ticket follow one-to-many cardinalities. Many-to-many relationships are resolved using associative tables like PresentedBy (Session–Speaker) and SponsoredBy (Event–Sponsor). Participation constraints ensure sessions cannot exist without an event, while sponsors may optionally be linked to events. This model accurately captures real-world event workflows

Database Design: Relational Schema



Query Execution: Event & Ticket Insights

Question 1: List all events with their venue names

```
SELECT Event.Title, Venue.Name AS VenueName
FROM Event
JOIN Venue ON Event.VenueID = Venue.VenueID;
```

Question 2: Find all sessions for a specific event (EventID = 5)

```
SELECT Title, StartTime, EndTime
FROM Session
WHERE EventID = 5;
```

Question 3: List all ticket types per event with prices

```
SELECT Event.Title, TicketType.Name, TicketType.Price
FROM TicketType
JOIN Event ON TicketType.EventID = Event.EventID;
```

Question 4: Count how many tickets were purchased for each event

```
SELECT Event.Title, COUNT(Ticket.TicketID) AS TicketsSold
FROM Ticket
JOIN TicketType ON Ticket.TicketTypeID = TicketType.TicketTypeID
JOIN Event ON TicketType.EventID = Event.EventID
GROUP BY Event.Title;
```


Query Execution: Attendee & Speaker Management

Question 5: Calculate the total revenue collected per event

```
SELECT Event.Title, SUM(Payment.Amount) AS TotalRevenue
FROM Payment
JOIN Ticket ON Payment.TicketID = Ticket.TicketID
JOIN TicketType ON Ticket.TicketTypeID = TicketType.TicketTypeID
JOIN Event ON TicketType.EventID = Event.EventID
GROUP BY Event.Title;
```

Question 7: Sponsors supporting more than one event

```
SELECT Sponsor.Name, COUNT(*) AS SponsoredEvents
FROM SponsoredBy
JOIN Sponsor ON SponsoredBy.SponsorID = Sponsor.SponsorID
GROUP BY Sponsor.Name
HAVING COUNT(*) > 1;
```

Question 9: Speakers presenting at Event 2

```
SELECT Speaker.SpeakerID, Speaker.Name
FROM Session
JOIN PresentedBy ON Session.SessionID = PresentedBy.SessionID
JOIN SessionSpeaker AS Speaker ON PresentedBy.SpeakerID = Speaker.SpeakerID
WHERE Session.EventID = 2;
```

Question 6: List attendees with unpaid tickets

```
SELECT Attendee.FirstName, Attendee.LastName, Ticket.TicketID
FROM Attendee
JOIN Ticket ON Attendee.AttendeeID = Ticket.AttendeeID
LEFT JOIN Payment ON Ticket.TicketID = Payment.TicketID
WHERE Payment.Status = 'Unpaid';
```

Question 8: List events organized by organizer 3

```
SELECT Title, StartDate, EndDate
FROM Event
WHERE OrganizerID = 3;
```

Question 10: Attendees who bought VIP tickets

```
SELECT FirstName, LastName
FROM Attendee
WHERE AttendeeID IN (
    SELECT AttendeeID
    FROM Ticket
    WHERE TicketTypeID IN (
        SELECT TicketTypeID
        FROM TicketType
        WHERE Name = 'VIP'
    )
);
```

Query Execution: Organizer & Sponsor Insights & Scheduling

Question 11: Event with the highest number of sold tickets

```
SELECT Event.Title, COUNT(Ticket.TicketID) AS TicketsSold
FROM Ticket
JOIN TicketType ON Ticket.TicketTypeID = TicketType.TicketTypeID
JOIN Event ON TicketType.EventID = Event.EventID
GROUP BY Event.Title
ORDER BY TicketsSold DESC
LIMIT 1;
```

Question 12: Events with no sponsors

```
SELECT Title
FROM Event E
WHERE NOT EXISTS (
    SELECT 1 FROM SponsoredBy S WHERE S.EventID = E.EventID
);
```

Question 13: Attendees who have not purchased any ticket

```
SELECT FirstName, LastName
FROM Attendee A
WHERE NOT EXISTS (
    SELECT 1 FROM Ticket T WHERE T.AttendeeID = A.AttendeeID
);
```

Question 14: Ticket type generating the highest revenue

```
SELECT TicketType.Name, SUM(Payment.Amount) AS Revenue
FROM Payment
JOIN Ticket ON Payment.TicketID = Ticket.TicketID
JOIN TicketType ON Ticket.TicketTypeID = TicketType.TicketTypeID
GROUP BY TicketType.Name
ORDER BY Revenue DESC
LIMIT 1;
```

Question 15: Sessions happening between 10 AM and 2 PM

```
SELECT Title, StartTime, EndTime
FROM Session
WHERE StartTime BETWEEN '10:00:00' AND '14:00:00';
```


Functional Dependencies

Functional dependencies are critical for ensuring the database design is normalized and maintains data integrity. Each dependency confirms that a specific attribute or set of attributes uniquely determines another attribute, preventing data redundancy and anomalies.

Event Details EventID → Title, Description, StartDate, EndDate, VenueID, OrganizerID	Venue Information VenueID → Name, Address, Capacity
Organizer Contact OrganizerID → Name, Email, Phone	Sponsor Data SponsorID → Name, ContactEmail
Ticket Type Specifics TicketTypeID → EventID, Name, Price, QuantityAvailable	Session Details SessionID → EventID, Title, Room, StartTime, EndTime
Speaker Information SpeakerID → Name, ContactEmail	Ticket Transaction TicketID → TicketTypeID, AttendeeID, PurchaseDate, Status
Attendee Profile AttendeeID → FirstName, LastName, Email, Phone	Payment Record PaymentID → TicketID, Amount, Method, PaymentDate, Status

Normalization Justification

All tables comply with 1NF, 2NF, and 3NF: - **1NF**: All attributes are atomic with no repeating groups. - **2NF**: Every non-key attribute fully depends on the primary key in all relations. - **3NF**: No transitive dependencies exist; attributes depend only on primary keys.This normalization ensures minimal redundancy, integrity, and efficient updates.

Relational Algebra Expressions

Relational algebra expressions provide a formal, theoretical foundation for database queries, illustrating the logical steps involved in data retrieval and manipulation. These expressions are fundamental to understanding how data is processed and combined within the relational model.



Tickets Sold Per Event

Calculates the total number of tickets sold for each event by joining the related tables and counting ticket records.

```
Event ⋈ TicketType ⋈  
Ticket γ Title,  
COUNT(TicketID) (Event)
```



Events with No Sponsors

Finds events that do not appear in the SponsoredBy table, meaning they have no sponsors.

```
Event -  
π(EventID)(SponsoredBy)
```



Attendees with No Tickets

Identifies attendees who have not purchased any tickets by subtracting attendees who appear in the Ticket table.

```
Attendee - π  
(AttendeeID)(Ticket)
```